1. **Sequência de LEDs estilo “Knight Rider”**

Usa 8 LEDs ligados ao porto E para criar um efeito de luz que se move da esquerda para a direita e depois de volta para a esquerda, repetindo indefinidamente.
A velocidade da animação pode ser controlada por um potenciômetro ligado a uma entrada analógica.

```c
#include <detpic32.h>

void delay(unsigned int ms);
unsigned int readADC();

int main(void) {
    TRISE = TRISE & 0xFF00;   // Configura RE0-RE7 como saída
    TRISBbits.TRISB4 = 1;     // Configura RB4 como entrada (AN4)

    // Configuração do ADC
    AD1PCFGbits.PCFG4 = 0;    // RB4 como entrada analógica
    AD1CON1bits.SSRC = 7;     // Modo de trigger automático
    AD1CON1bits.CLRASAM = 1;  // Para conversões após a 1ª interrupção
    AD1CON3bits.SAMC = 16;    // Tempo de amostragem = 16 TAD
    AD1CON2bits.SMPI = 0;     // Interrupção após 1 amostra
    AD1CHSbits.CH0SA = 4;     // Entrada analógica AN4
    AD1CON1bits.ON = 1;       // Ativa o ADC

    unsigned char led = 0x01;
    int direction = 1;

    while (1) {
        LATE = (LATE & 0xFF00) | led;  // Atualiza os LEDs

        unsigned int speed = 10 + (readADC() * 490) / 1023; // Controle de velocidade ajustado
        delay(speed);

        if (direction == 1) {
            led <<= 1;
            if (led == 0x80) direction = -1;
        } else {
            led >>= 1;
            if (led == 0x01) direction = 1;
        }
    }

    return 0;
}

unsigned int readADC() {
    AD1CON1bits.ASAM = 1;  // Inicia a conversão
    while (IFS1bits.AD1IF == 0);  // Espera pelo fim da conversão
    IFS1bits.AD1IF = 0;  // Reseta flag da interrupção
    return ADC1BUF0;  // Retorna o valor lido
}

void delay(unsigned int ms) {
    resetCoreTimer();
    while (readCoreTimer() < 20000 * ms);
}
```

2. **ADD Modulo 50**

Contador de modulo 50, que acrescenta conforme o press do int 1. Nos LED's deve aparecer a conversão em BCD

LED	LED7 ... LED0	RE7 ... RE0
Switches	DS4 ... DS1	RB3 ... RB0
Segmentos dos *displays*	Seg G ... Seg A	RB14 ... RB8
Controlo dos *displays*	*Display* mais significativo, *Display* menos significativo	RD6, RD5
Entrada analógica 0 a 3.3V	Potenciómetro (R30)	RB4
Frequência dos sinais de relógio	*Core Timer*, PBCLK	20MHz

```c
#include <detpic32.h>

void send2displays(unsigned char value);
unsigned char toBCD(unsigned char value);
void delay(unsigned int ms);

int main(void) {
    TRISE = TRISE & 0xFF00;       // RE0-RE7 como saída (LEDs)
    TRISDbits.TRISD8 = 1;         // RD8 como entrada (botão)
    TRISB = TRISB & 0x80FF;       // RB8-RB14 como saída (displays)
    TRISDbits.TRISD5 = 0;
    TRISDbits.TRISD6 = 0;
    CNPUEbits.CNPUE8 = 1;         // Habilita pull-up no botão RD8
    LATE = LATE & 0xFF00;         // Limpa os bits dos LEDs


    int cnt = 0;
    int lastState = PORTDbits.RD8;

    while (1) {
        int currentState = PORTDbits.RD8;

        if (currentState == 0 && lastState == 1) { // Detecção de borda de descida
            delay(50); // Debounce
            if (PORTDbits.RD8 == 0) { // Confirma leitura estável
                cnt = (cnt + 1) % 50;
                LATE = (LATE & 0xFF00) | toBCD(cnt); // Enviar valor em BCD para LEDs
            }
        }

        send2displays(toBCD(cnt));
        delay(0); // Pequeno atraso para multiplexagem
        lastState = currentState;
    }
    return 0;
}

void send2displays(unsigned char value) {
    static const char disp7Scodes[] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07,
                                       0x7F, 0x6F, 0x77, 0x7C, 0x39, 0x5E, 0x79, 0x71};
    static char displayFlag = 0;

    int digit_high = value >> 4;
    int digit_low = value & 0x0F;

    if (displayFlag == 0) {
        LATB = (LATB & 0x80FF) | (disp7Scodes[digit_high] << 8);
        LATDbits.LATD5 = 0;  // Ativar display esquerdo
        LATDbits.LATD6 = 1;  // Desativar display direito
    } else {
        LATB = (LATB & 0x80FF) | (disp7Scodes[digit_low] << 8);
        LATDbits.LATD5 = 1;  // Desativar display esquerdo
        LATDbits.LATD6 = 0;  // Ativar display direito
    }

    displayFlag = !displayFlag;  // Alternar flag para multiplexagem
    delay(5);  // Pequeno atraso para garantir atualização correta
}

void delay(unsigned int ms) {
    resetCoreTimer();
    while (readCoreTimer() < 20000 * ms);
}

unsigned char toBCD(unsigned char value){
    return (((value / 10) << 4) + (value % 10));
}
```

Exercício 1

a) Escreva um programa em linguagem *assembly* que, em ciclo infinito, faça a leitura dos 4 bits do *dip-switch* da placa DETPIC32 (ligado aos portos RB3 a RB0) e mostre o valor de cada um deles nos 4 LED ligados aos portos RE3 a RE0. Não se esqueça de fazer todas as configurações necessárias.

b) Altere o programa que escreveu na alínea a), de modo a que o valor de cada um dos bits do *dip-switch* seja mostrado por ordem inversa nos 4 LED (i.e. RB3->RE0, RB2->RE1, ..., RB0->RE3).

c) Faça uma nova alteração ao programa que escreveu em a), de modo a que o valor de cada um dos bits do *dip-switch* seja mostrado por ordem inversa nos 4 LED ligados aos 4 bits mais significativos do porto E (i.e. RB3->RE4, ..., RB0->RE7); os 4 LED ligados aos 4 bits menos significativos do porto E devem mostrar o valor presente nas entradas RB3-RB0 (i.e. RB3->RE3, ..., RB0->RE0).

a)

```
#include <detpic32.h>

int main (void){

    TRISB = TRISB | 0x000F;                 //dip-switches INPUT's
    TRISE = TRISE & 0xFFF0;                 //LED3- LED0 OUTPUT's
    LATE = LATE & 0x0000;                   //clear aos leds
    while(1){

        LATEbits.LATE0 = PORTBbits.RB0;
        LATEbits.LATE1 = PORTBbits.RB1;
        LATEbits.LATE2 = PORTBbits.RB2;
        LATEbits.LATE3 = PORTBbits.RB3;
    }



}
```

b)

```
#include <detpic32.h>

int main (void){

    TRISB = TRISB | 0x000F;                 //dip-switches INPUT's
    TRISE = TRISE & 0xFFF0;                 //LED3- LED0 OUTPUT's
    LATE = LATE & 0x0000;                   //clear aos leds
    while(1){

        LATEbits.LATE0 = PORTBbits.RB0;
        LATEbits.LATE1 = PORTBbits.RB1;
        LATEbits.LATE2 = PORTBbits.RB2;
        LATEbits.LATE3 = PORTBbits.RB3;
    }



}
```

c)

```
#include <detpic32.h>

int main (void){

    TRISB = TRISB | 0x000F;                 //dip-switches INPUT's
    TRISE = TRISE & 0xFF00;                 //LED7- LED0 OUTPUT's
    LATE = LATE & 0x0000;                   //clear aos leds
    while(1){

        LATEbits.LATE0 = PORTBbits.RB0;
        LATEbits.LATE1 = PORTBbits.RB1;
        LATEbits.LATE2 = PORTBbits.RB2;
        LATEbits.LATE3 = PORTBbits.RB3;

        LATEbits.LATE4 = PORTBbits.RB3;
        LATEbits.LATE5 = PORTBbits.RB2;
        LATEbits.LATE6 = PORTBbits.RB1;
        LATEbits.LATE7 = PORTBbits.RB0;
    }



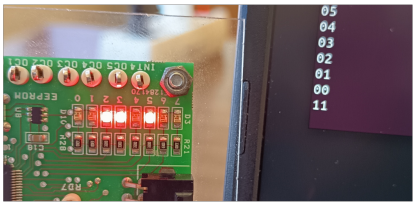
}
```

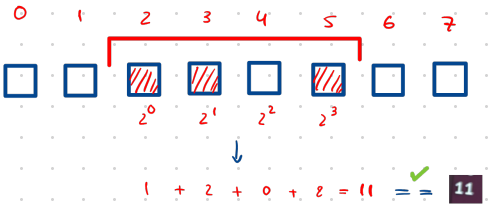
Exercicio 2

Escreva e teste um programa, em linguagem C, que faça todas as configurações necessárias e que mostre nos 4 LED ligados aos portos RE5 a RE2, o valor de um contador decrescente módulo 12 (valor inicial: 0), a frequência de atualização do contador deve depender, continuamente, do estado lógico do "switch" DS3 (ligado ao porto RB2): DS3 OFF, f=2.3Hz; DS3 ON, f=5.5Hz.

O valor do contador deve também ser mostrado no ecrã do PC, em decimal formatado com 2 dígitos (os valores devem ser apresentados em linhas consecutivas do ecrã).

O valor da frequência deve ser obtido com o menor erro possível. Edite o seu programa no ficheiro **t1.c**.






```c
#include <detpic32.h>

void delay(unsigned int ms);

int main(void) {
    // Configuração dos LEDs (RE5 a RE2 como saída)
    TRISE = TRISE & 0xFFC3;
    LATE = LATE & 0xFFC3;

    // Configuração do switch (RB2 como entrada)
    TRISBbits.TRISB2 = 1;

    int cnt = 0;  // Inicializa o contador com 0 para descer até 0 módulo 12
    unsigned int freq;

    while (1) {
        // Verifica o estado do switch e ajusta a frequência
        if (PORTBbits.RB2 == 0)
            freq = 435;  // 2.3Hz (1000ms / 2.3 ≈ 435ms por ciclo)
        else
            freq = 182;  // 5.5Hz (1000ms / 5.5 ≈ 182ms por ciclo)

        // Atualiza os LEDs com o valor do contador
        LATE = (LATE & 0xFFC3) | (cnt << 2);

        // Mostra o valor do contador no ecrã do PC
        printf("%02d\n", cnt);

        // Aguarda o tempo correspondente à frequência
        delay(freq);
        // Atualiza o contador
        if (cnt == 0) {
            cnt = 11;
        } else {
            cnt = cnt - 1;
        }
    }

    return 0;
}

void delay(unsigned int ms) {
    resetCoreTimer();
    while (readCoreTimer() < 20000 * ms);
}
```

Exercicio 2

Escreva e teste um programa, em linguagem C, que faça todas as configurações necessárias e que converta o sinal na entrada do módulo A/D (AN4/RB4), à frequência de 6Hz, com 2 conversões por cada amostra, e que:

i) mostre no ecrã do PC a média dos resultados da conversão, em hexadecimal formatado com 3 dígitos (os valores devem ser apresentados em linhas consecutivas do ecrã);
ii) apresente uma medida do valor obtido, numa escala 0 a 9, no display menos significativo da placa;
iii) a cada nova conversão mude o estado do LED1, ligado ao porto RE1 (valor inicial: RE1=0).

Apenas detecção do fim de conversão por **polling**. Edite o seu programa no ficheiro t2.c.

```c
#include <detpic32.h>

void delay(unsigned int ms);

int main(void) {
    // Configuração do ADC para AN4 (RB4)
    TRISBbits.TRISB4 = 1;       // RB4 como entrada
    AD1PCFGbits.PCFG4 = 0;      // AN4 configurado como entrada analógica
    AD1CON1bits.SSRC = 7;       // Disparo automático
    AD1CON1bits.CLRASAM = 1;    // Parar conversões após a interrupção
    AD1CON3bits.SAMC = 16;      // Tempo de amostragem: 16 TAD (1.6 µs)
    AD1CON2bits.SMPI = 1;       // 2 amostras antes da interrupção (N-1)
    AD1CHSbits.CH0SA = 4;       // Selecionar AN4 (RB4)
    AD1CON1bits.ON = 1;         // Ativar o ADC

    while (1) {
        // Iniciar conversão
        AD1CON1bits.ASAM = 1;

        // Esperar que a conversão termine
        while (!IFS1bits.AD1IF);
        IFS1bits.AD1IF = 0; // Reset da flag de interrupção

        // Ler os valores convertidos
        int amostra1 = ADC1BUF0;
        int amostra2 = ADC1BUF1;

        // Calcular a média das duas amostras
        int media = (amostra1 + amostra2) / 2;

        // Mostrar no PC em hexadecimal (3 dígitos)
        printf("%03X\n", media);

        // Aguardar 166ms para garantir 6 Hz de amostragem
        delay(166);
    }

    return 0;
}

void delay(unsigned int ms) {
    resetCoreTimer();
    while (readCoreTimer() < 20000 * ms);
}
```

i)

amostras → ADC1BUF0 até ADC1BUF3

$T = \frac{1}{f}$

$= \frac{1}{6} = 0,16666$ segundos

$= 166,6$ ms

$\simeq 166$ ms

↘ delay(166);

ii)

```c
void delay(unsigned int ms);
void sendToDisplay(unsigned char value);

int main(void) {
    // Configuração do ADC para AN4 (RB4)
    TRISBbits.TRISB4 = 1;       // RB4 como entrada
    AD1PCFGbits.PCFG4 = 0;      // AN4 configurado como entrada analógica
    AD1CON1bits.SSRC = 7;       // Disparo automático
    AD1CON1bits.CLRASAM = 1;    // Parar conversões após a interrupção
    AD1CON3bits.SAMC = 16;      // Tempo de amostragem: 16 TAD (1.6 µs)
    AD1CON2bits.SMPI = 1;       // 2 amostras antes da interrupção (N-1)
    AD1CHSbits.CH0SA = 4;       // Selecionar AN4 (RB4)
    AD1CON1bits.ON = 1;         // Ativar o ADC

    // Configuração dos displays de 7 segmentos
    TRISB = TRISB & 0x80FF;     // Configurar RB8 a RB14 como saídas
    TRISD = TRISD & 0xFF9F;     // Configurar RD5 e RD6 como saídas

    while (1) {
        // Iniciar conversão
        AD1CON1bits.ASAM = 1;

        // Esperar que a conversão termine
        while (!IFS1bits.AD1IF);
        IFS1bits.AD1IF = 0; // Reset da flag de interrupção

        // Ler os valores convertidos
        int amostra1 = ADC1BUF0;
        int amostra2 = ADC1BUF1;

        // Calcular a média das duas amostras
        int media = (amostra1 + amostra2) / 2;

        // Escalar para valores entre 0 e 9
        int displayValue = (media * 9) / 1023;

        // Mostrar no display de 7 segmentos
        sendToDisplay(displayValue);

        // Mostrar no PC em hexadecimal (3 dígitos)
        printf("%03X\n", media);

        // Aguardar 166ms para garantir 6 Hz de amostragem
        delay(166);
    }

    return 0;
}

// Função para gerar atraso em milissegundos
void delay(unsigned int ms) {
    resetCoreTimer();
    while (readCoreTimer() < 20000 * ms);
}

// Função para enviar o valor para o display de 7 segmentos
void sendToDisplay(unsigned char value) {
    static const unsigned char displayCodes[10] = {
        0x3F, // 0
        0x06, // 1
        0x5B, // 2
        0x4F, // 3
        0x66, // 4
        0x6D, // 5
        0x7D, // 6
        0x07, // 7
        0x7F, // 8
        0x6F  // 9
    };

    LATB = (LATB & 0x80FF) | (displayCodes[value] << 8);  // Enviar código para RB8-RB14
    LATD = (LATD & 0xFF9F) | 0x0020;  // Ativar o display menos significativo (D1)
}
```

Escala de 0 - 9

value = displayValue

ou:

```c
LATDbits.LATD5 = 1;
```